

Home assignment - Research Analyst Upwind

By Eyal Subeli

Part 1 : eviction_attack ,pages 2-3

Part 2 : iam_policy_classifier, pages 4-6

part 3 : pages 7-12

Part 1-eviction_attack

Vulnerability Summary:

The original eviction algorithm relies on a fully deterministic computation of a cyclical distance between an incoming virtual page and all pages currently resident in physical memory. On each memory access, the algorithm evaluates this distance for every resident page and evicts the page that maximizes it, with ties consistently resolved using a fixed rule based on the lowest physical frame index.

This strict determinism introduces a fundamental security weakness. The eviction decision is a pure function of the requested virtual page identifier and the current memory state both of which are either directly controllable or reliably observable by a malicious tenant. As a result, an attacker capable of issuing memory access requests can deliberately craft an input that deterministically forces the eviction of a specific target page, with no uncertainty.

Since the virtual address space is modeled as a ring of size NUM_PAGES, the attacker can analytically compute the virtual page located at the diametrically opposite point on the ring, namely:

$$(target_{page} + \frac{num_{pages}}{2}) \bmod num_{pages}$$

This page maximizes the cyclical distance to the target page and therefore guarantees that the target will be selected for eviction by the algorithm. Importantly, even without explicit knowledge of the eviction formula, a black-box attacker can infer this behavior through systematic probing and observation of eviction outcomes, ultimately achieving the same effect.

The ability to reliably evict a chosen page enables a targeted eviction attack. In a multi-tenant cloud environment, such behavior may translate into a denial-of-service condition, for example by repeatedly evicting pages containing critical data, degrading system performance, or interfering with co-located tenants sharing the same physical RAM. In the simulated environment, this vulnerability was empirically demonstrated by evicting the protected target page (Page 50) with a 100% success rate, confirming that determinism in eviction policies can act as a security liability rather than a benign design choice.

Mitigation Logic:

To prevent reliable targeted eviction of a specific page, as demonstrated in the exploit section, the determinism of the eviction algorithm must be reduced without significantly degrading performance. Rather than replacing the original eviction heuristic, the goal is to introduce **controlled unpredictability** that prevents an attacker from precisely predicting eviction outcomes.

The proposed mitigation is **Top-K Randomization**. The algorithm continues to compute the cyclical distance between the incoming page and all resident pages, thereby preserving the original locality-aware eviction intuition. However, instead of deterministically evicting the single page with the maximal distance, it randomly selects a victim from the top K pages with the highest distances.

By restricting randomization to this high-distance set, memory locality is largely preserved, while the eviction decision itself becomes non-deterministic. For small values of K , the behavior remains close to the original policy, but some residual structure remains, allowing an attacker to partially predict which pages are more likely to be evicted. As K increases, the eviction choice is drawn from a broader candidate set, significantly increasing attacker uncertainty and preventing reliable prediction of the evicted page. This added randomness comes at the cost of gradually diverging from the original deterministic behavior, potentially affecting locality and performance. Several alternative mitigation strategies were also evaluated, including SoftMax-based eviction, ϵ -greedy selection, and noise-perturbed distance ranking. While these approaches reduced predictability, they either introduced higher performance variance or deviated more substantially from the original eviction behavior.

In practice, $K = 3$ was found to provide a balanced operating point: it is large enough to break deterministic eviction guarantees and prevent reliable targeted eviction attacks, while remaining small enough to preserve performance characteristics nearly identical to the original algorithm, as confirmed by the benchmark results.

Results:

Performance Benchmark:

The performance impact of the proposed mitigations was evaluated using the Page Fault Rate (PFR), defined as the ratio between memory misses and total accesses.

Each policy was tested over **1,000 random accesses**, repeated across **6 different random seeds** using identical access traces.

Table X reports the mean PFR and standard deviation for the deterministic baseline and all evaluated eviction policies.

Std. Deviation (Misses / Total Accesses)	Mean Page Fault Rate (Misses / Total Accesses)	Parameters	Eviction Policy
0.0046	0.8983	-	Deterministic Max-Distance
0.0055	0.8947	$K=3$	Top-K Randomized
0.0100	0.9003	$T = 1.0$	Softmax-Based
0.0046	0.9025	$\epsilon = 0.05$	ϵ -Greedy
0.0047	0.9008	$\sigma = 0.1$	Noise-Perturbed Ranking

Analysis:

- **Performance Impact:** The results indicate that introducing controlled entropy does **not** cause a significant increase in the Page Fault Rate. The observed average difference (Mean Δ) is approximately 0.0037 in absolute Page Fault Rate (≈ 0.37 percentage points), which is negligible from a performance perspective.
- **Stability:** The low standard deviation (≈ 0.0024) demonstrates that the revised algorithm maintains stable and predictable performance across different random seeds, despite the introduction of randomized eviction decisions.
- **Security vs. Efficiency:** While performance remains **almost identical** to the original deterministic policy, the security posture improves substantially. Under the original algorithm, the target page was evicted successfully in **100%** of attack attempts. In contrast, under Top-K randomization, eviction outcomes are no longer deterministic, preventing the attacker from reliably predicting or forcing the eviction of a specific target page and effectively blocking the targeted eviction attack vector.

Part 2 - iam_policy_classifier

Methodology:

The goal of this experiment was to evaluate the ability of different Large Language Models (LLMs) to classify AWS IAM policies as either **Weak** or **Strong** from a security perspective.

The system was implemented as a Python-based classification engine that accepts IAM policies in JSON format, sends them to an LLM for analysis, and returns a structured JSON response containing the original policy, a classification label (Weak / Strong), and a short security-focused reasoning.

Two representative IAM policies were used:

- **Weak policy:** broad permissions using wildcards (` Action: "\*"`, Resource: "*"`) without restrictive conditions.
- **Strong policy:** Scoped permissions with a specific action and resource, protected by a strong contextual security condition (mandatory MFA).

Each policy was evaluated using multiple LLM providers and models:

- OpenAI (GPT-4o-mini)
- Ollama (phi)
- Ollama (llama3.2)

All experiments were executed in a single automated run, and the results were collected into structured CSV and JSON files to enable direct comparison across models.

Prompt Iteration and Improvement Process:

The initial prompt provided the LLM with a high-level instruction to classify IAM policies as either *Weak* or *Strong* based on general security best practices, without strict decision rules or output constraints.

Initial Prompt Behavior

Early prompt versions exhibited several inconsistencies:

- Some models returned explanations or additional text outside the JSON schema.
- Policies involving destructive actions were sometimes misclassified due to overly generic decision rules.
- Output formats varied between models, especially in local LLMs.

Baseline Prompt Results:

```
PS C:\Users\eyals\MyProjects\Upwind> python -m iam_policy_classifier.run_demo
DEBUG OPENAI_API_KEY: True
IAM Policy Classification - Stage 3 Experiment Runner

-----
Running: policy=weak | provider=openai | model=gpt-4o-mini
Running: policy=weak | provider=ollama | model=phi:latest
Running: policy=weak | provider=ollama | model=llama3.2:latest
Running: policy=strong | provider=openai | model=gpt-4o-mini
Running: policy=strong | provider=ollama | model=phi:latest
Running: policy=strong | provider=ollama | model=llama3.2:latest

Experiment Results Summary
-----
Policy    Provider  Model          Result
-----
weak      openai    gpt-4o-mini    Weak
weak      ollama    phi:latest      TIMEOUT
weak      ollama    llama3.2:latest Weak
strong    openai    gpt-4o-mini    Weak
strong    ollama    phi:latest      Weak
strong    ollama    llama3.2:latest Strong
-----
```

Refined Prompt Design (Version 2)

Following the inconsistencies observed with the initial prompt, a refined version was introduced to transform the classification task from a general LLM judgment into a deterministic, security-focused IAM evaluation.

The primary goal of this refinement was to reduce ambiguity, standardize decision logic, and ensure consistent behavior across different LLM providers.

Key Improvements Introduced

- **Security-Oriented Role Definition:** The prompt explicitly defines the model as a Senior Cloud Security Expert, anchoring the analysis in real-world IAM risk assessment practices and promoting conservative, risk-aware decisions.
- **Explicit Evaluation Criteria:** Clear security principles were introduced, including least privilege enforcement, permission scope analysis, contextual conditions (such as MFA), and blast radius assessment. This ensures that policies are evaluated using a consistent and structured framework.
- **Deterministic Decision Rules:** Common IAM risk patterns were explicitly defined, including the handling of wildcard permissions, destructive actions, unconditional Allow statements, and explicit Deny rules. This significantly reduced model-dependent interpretation differences.
- **Strict Output Constraints:** The refined prompt enforces a fixed JSON-only output schema, restricts the classification values to Weak or Strong, and prohibits any additional explanations or formatting. These constraints stabilized output formats and enabled reliable validation.
- **Conservative Tie-Breaking:** In cases of uncertainty, the prompt instructs the model to choose the most security-conservative classification, aligning model behavior with standard security review practices.

Results Summary:

Baseline Policies – Results Summary			
Policy	Provider	Model	Result
weak_baseline	openai	gpt-4o-mini	Weak
weak_baseline	ollama	phi:latest	Weak
weak_baseline	ollama	llama3.2:latest	Weak
strong_baseline	openai	gpt-4o-mini	Strong
strong_baseline	ollama	phi:latest	Weak
strong_baseline	ollama	llama3.2:latest	Strong

The results show improved classification consistency across models. GPT-4o-mini correctly classifies both the weak and strong baseline policies, aligning with the expected security assessment. In contrast, phi: latest still misclassifies the strong baseline policy as “Weak”, indicating that the smaller local model remains more sensitive to destructive actions and may underweight contextual constraints (e.g., scope/MFA) compared to the stronger model.

Relevant files: baseline_results.csv , baseline_results.json

Validation and Dataset Expansion

To further validate the robustness of the refined prompt, the evaluation process was extended beyond the initial baseline policies.

At this stage, the dataset was deliberately expanded to reduce overfitting to a small number of handcrafted examples and to better assess prompt behavior under varied policy structures.

Synthetic Policy Generation

A dedicated generation agent was introduced to automatically create additional IAM policy samples labeled as *Weak* and *Strong*.

These generated policies varied in permission scope, action types, resource definitions, and contextual conditions, enabling broader coverage of common IAM risk patterns.

The purpose of this step was to ensure that the refined prompt generalizes correctly across a wider range of inputs and does not rely on memorizing specific policy structures.

Prompt Validation on Expanded Dataset

The generated policies were classified using the refined prompt, and the results were analyzed for consistency, correctness, and output stability.

This process increased confidence that the prompt behaves deterministically and applies the intended security logic across diverse inputs.

External Validation Using Google AI Studio

In addition to local and scripted evaluations, prompt behavior was manually validated using **Google AI Studio**.

This external validation step was used to:

- Cross-check classification behavior in a separate inference environment
- Verify that the refined decision logic holds outside the local execution setup
- Identify any environment-specific deviations or unexpected outputs

Validation Results

The refined prompt was evaluated against an expanded dataset of synthetically generated IAM policies labeled as Weak and Strong.

The full classification results, including per-policy reasoning across all tested models, are provided in the attached CSV file.

Overall, the results demonstrate that the refined prompt applies consistent security logic across a broad set of inputs, while also highlighting clear capability differences between models.

GPT-4o-mini showed the highest alignment with the expected classifications, correctly identifying over-permissive policies as Weak and conditionally restricted destructive policies as Strong in most cases.

In contrast, the smaller phi model frequently misclassified strong policies as Weak, particularly when destructive actions were involved, even when protected by MFA or narrow scope constraints.

The llama3.2 model demonstrated intermediate behavior, correctly classifying most strong policies but still exhibiting occasional conservative bias in cases involving wildcard resources. These results confirm that the refined prompt significantly improves determinism and security alignment, while also illustrating that model capacity plays a critical role in correctly interpreting contextual IAM constraints.

Relevant files: generated_results.csv, generated_results.json

Notes and Limitations

- Further validation is required on a broader set of synthetically generated IAM policies to fully assess the robustness of the refined prompt.
- Additional focus is needed on *Strong* generated policies, which are inherently more complex and rely on correct interpretation of contextual constraints such as MFA enforcement and resource scoping.
- Some classification discrepancies are attributable to model-level limitations rather than prompt design alone.
- Smaller local models are constrained by limited compute capacity and inference time, which can reduce their ability to consistently reason over complex IAM policies involving destructive actions and conditional enforcement.
- While the refined prompt improves determinism and security alignment, model capability remains a key factor in accurately evaluating nuanced IAM configurations.

Part 3:

Case 1:

Description: *The security group/firewall allows inbound access from 0.0.0.0/0 to administrative or data-plane ports (e.g., SSH/RDP/DB). This creates an Internet-exposed attack surface that enables automated scanning, brute-force attempts, and exploitation of vulnerabilities, increasing the likelihood of host compromise and lateral movement within the cloud environment.*

Why it is Critical: This misconfiguration bypasses the "Least Privilege" networking principle. It exposes internal services directly to the public internet, making them vulnerable to zero-day exploits and credential stuffing even before any application-level security is applied.

risk assessment:

- **Unauthorized Access & Host Compromise:** Internet-exposed administrative and data-plane ports allow attackers to gain unauthorized access to critical hosts, potentially resulting in full system compromise and persistent control.
- **Data Exposure, Exfiltration & Ransomware:** A compromised host may be leveraged to access sensitive data stores, exfiltrate confidential information, or encrypt critical systems and databases, causing both data loss and service unavailability.
- **Lateral Movement & Cloud Identity Abuse:** Once initial access is obtained, attackers can pivot to internal resources and abuse cloud identity mechanisms, including metadata services, to escalate privileges and expand their reach across the cloud environment.
- **Automated Internet Threats & Operational Impact:** Continuous exposure to automated scanning, brute-force attempts, and bot-driven activity increases resource consumption, operational noise, and delays in detecting targeted attacks.
- **Internal & External Supply Chain Risk:** If the exposed asset participates in build or CI/CD processes, a compromise could enable injection of malicious code into internal pipelines, potentially impacting downstream systems and customers.

Consolidated Risk Assessment:

Unrestricted network access to administrative and data-plane ports exposes critical cloud assets to unauthorized access and host compromise. A successful breach may lead to data exfiltration, ransomware-driven service disruption, and lateral movement across the cloud environment through abused identity and metadata services. In addition, continuous exposure to automated internet-based threats increases operational overhead and obscures detection of targeted attacks. If the exposed asset participates in build or CI/CD workflows, the risk further extends to internal and external supply chain compromise. Overall, this misconfiguration represents a **High to Critical risk** due to its broad attack surface, high likelihood of exploitation, and severe potential impact.

Recommended Remediation Strategy:

To mitigate the risks associated with unrestricted network access, inbound exposure to administrative and data-plane ports should be removed and restricted to trusted network boundaries only. Management access must be performed through secured channels such as VPNs, bastion hosts, or identity-aware proxies, rather than direct internet-facing connections. Sensitive services should be isolated within private network segments and exposed only through authenticated gateways or load balancers. In addition, cloud identity protections should be enforced by limiting instance-level permissions and securing access to metadata services. Finally, continuous monitoring and policy-based controls should be implemented to detect and prevent reintroduction of overly permissive network configurations.

Preventive Network Guardrails:

Beyond remediating existing firewall and security group rules, organizations should enforce preventive, policy-based guardrails that block the creation or modification of inbound rules allowing 0.0.0.0/0 access to administrative or data-plane ports.

These controls should be applied at the infrastructure provisioning or configuration layer to ensure overly permissive network exposure cannot be introduced, even unintentionally, during future changes or deployments.

Verification and Ongoing Validation:

To verify remediation, security groups and firewall rules should be audited to ensure that no administrative or data-plane ports are exposed to 0.0.0.0/0. External network scans can be performed to confirm that sensitive ports are no longer accessible from the public internet.

Ongoing validation should include continuous monitoring of network configurations using cloud-native posture management tools, policy-as-code enforcement for infrastructure changes, and alerting on newly introduced overly permissive firewall rules. Network flow logs and authentication logs can be leveraged to detect anomalous access attempts and validate that management access occurs only through approved channels.

Case 2:

Description:

An internal or customer-facing application endpoint is exposed without proper authentication and authorization controls, allowing anonymous users to access sensitive functionality, internal APIs, or administrative interfaces. As a result, any external party can interact directly with application logic that was not designed to be publicly accessible, potentially bypassing business rules, access controls, and security assumptions.

This exposure may affect REST APIs, web interfaces, background service endpoints, or administrative routes that rely on implicit trust in the network or assume authenticated users by default.

Why it is Critical:

This issue is critical because it removes identity as a security boundary at the application layer. Without authentication or authorization enforcement, the application cannot distinguish between legitimate users and attackers, enabling unauthorized access to sensitive functionality regardless of network-level protections.

Unlike network-level misconfigurations, unauthenticated application exposure directly impacts business logic and data handling paths. Attackers can invoke internal APIs, access administrative functions, or manipulate application state without exploiting vulnerabilities, making the attack low-effort and high-impact. In addition, such activity may bypass perimeter defenses and remain undetected, as malicious requests can appear similar to legitimate application traffic.

risk assessment:

- **Unauthorized Functionality & Business Logic Abuse:** Anonymous access enables attackers to invoke sensitive application functionality and manipulate business workflows that were intended to be restricted to authenticated users.
- **Sensitive Data Exposure & Enumeration:** Unprotected endpoints may allow unauthorized retrieval, enumeration, or inference of sensitive user and business data through legitimate application interfaces.
- **Administrative Control & System Manipulation:** If administrative or internal endpoints are exposed, attackers may alter configurations, trigger destructive actions, or disrupt service availability without authentication.
- **Logical Privilege Escalation:** Missing authorization checks can allow attackers to perform actions beyond intended access levels by abusing application logic rather than exploiting technical vulnerabilities.
- **Operational, Compliance, and Reputational Impact:** The lack of identity enforcement eliminates audit trails, increases regulatory exposure, and undermines user trust, amplifying the business impact of any successful abuse.

Consolidated Risk Assessment:

Unauthenticated application exposure allows anonymous users to directly interact with sensitive application functionality, bypassing identity and access controls at the application layer. This significantly increases the risk of unauthorized actions, data exposure, and manipulation of business logic without requiring exploitation of technical vulnerabilities. Attackers may leverage unrestricted access to internal APIs or administrative endpoints to escalate impact, disrupt operations, or abuse system resources at scale. In addition, the absence of authentication removes accountability and auditability, increasing compliance risk and complicating detection and incident response. Overall, this issue represents a High to Critical risk due to its low barrier to exploitation, broad potential impact, and direct exposure of core application logic.

Recommended Remediation Strategy:

Authentication and authorization should be enforced by default, ensuring that sensitive functionality is only accessible to verified identities with explicitly defined permissions and

eliminating implicit trust at the application layer. Access to application APIs should be centralized through gateways or reverse proxies that consistently enforce authentication, authorization, and rate limiting. Administrative and internal endpoints must be isolated from public access and protected with stronger controls such as multi-factor authentication and identity-aware proxies. Finally, continuous monitoring and automated validation should be implemented to ensure that newly deployed endpoints are not exposed without proper access controls, preventing regression and maintaining secure-by-design application practices.

Verification and Ongoing Validation:

To verify remediation, application endpoints should be tested to ensure that all sensitive APIs and interfaces enforce authentication and authorization, and that unauthenticated requests are rejected. Automated API testing and security testing tools can be used to confirm that no endpoints are accessible without valid credentials.

Ongoing validation should include continuous monitoring of application access logs to detect unauthorized or anonymous requests, as well as integration of authentication checks into CI/CD pipelines to prevent deployment of unprotected endpoints. API gateways, application firewalls, and security testing frameworks can help enforce consistent access controls and detect regressions over time.

Case 3:

Description:

Build artifacts, container images, or packaged application binaries include embedded Git access tokens or credentials that were inadvertently hardcoded or injected during the build process. These artifacts may be stored in container registries, artifact repositories, or deployed across environments, allowing anyone with access to them to extract the embedded credentials. As a result, unauthorized parties may gain access to private source code repositories, version control systems, or CI/CD pipelines, even without direct access to the original development environment or source code.

Why it is Critical:

This issue is critical because embedded Git tokens allow attackers to authenticate directly as trusted systems or developers, bypassing perimeter and identity controls protecting the source code and build infrastructure. Unlike source-level secret exposure, credentials embedded in artifacts often persist across multiple environments and versions, significantly increasing the blast radius and duration of the exposure.

Compromised Git credentials can be used to modify source code, tamper with build pipelines, or inject malicious code into downstream releases, escalating a single secret leak into a full supply chain compromise.

risk assessment:

- **Unauthorized Source Code & Intellectual Property Exposure:** Embedded credentials allow attackers to access private repositories, exposing proprietary source code, internal system design, and sensitive implementation details that facilitate further attacks.
- **Secret Amplification & Identity Impersonation:** Compromised Git tokens often lead to discovery of additional secrets and enable attackers to impersonate trusted developer or automation identities, bypassing authentication safeguards and expanding access across environments.
- **CI/CD Pipeline Takeover & Supply Chain Compromise:** Write-capable or administrative tokens can be abused to modify source code or CI/CD workflows, injecting malicious changes that propagate through automated builds and impact downstream systems or customers.
- **Persistent Access & Incident Response Complexity:** The long-lived and widely distributed nature of build artifacts allows compromised credentials to remain exploitable over time, increasing remediation effort, regulatory exposure, and overall incident severity.

Consolidated Risk Assessment:

Embedded Git access tokens within build artifacts, container images, or packaged binaries expose critical development and build infrastructure to unauthorized access. Attackers who extract these credentials can authenticate as trusted systems or developers, bypassing identity and perimeter controls to access private repositories, CI/CD pipelines, and related resources. This exposure enables escalation from source code theft to widespread secret leakage, pipeline manipulation, and supply chain compromise affecting downstream systems and customers. Due to the persistent nature of artifacts and the difficulty of fully tracking their distribution, such incidents significantly complicate detection, remediation, and incident response. Overall, this issue represents a **High to Critical risk** because of its potential for long-term abuse, broad blast radius, and severe impact on both internal operations and external trust.

Recommended Remediation Strategy:

To mitigate the risk of embedded Git tokens, secrets must never be included in build artifacts, container images, or packaged binaries. All credentials should be managed through dedicated secret management solutions and injected only at runtime, with strict separation between build-time and runtime access. Git tokens should follow the principle of least privilege, use scoped or short-lived permissions, and be rotated immediately upon exposure. CI/CD pipelines must be hardened to prevent credential leakage, and automated secret scanning should be integrated into build and artifact workflows to detect and prevent regression. Finally, organizations should maintain rapid response processes for credential revocation, artifact invalidation, and auditing of repository and pipeline access in order to minimize blast radius in the event of exposure.

Developer Workstation Protections:

In addition to CI/CD-level controls, organizations should **also** deploy developer environment protections such as IDE extensions or pre-commit checks designed to detect hardcoded secrets and access tokens. These tools help prevent credentials from leaving the developer's workstation by blocking commits or generating alerts before secrets reach source control systems, build pipelines, or artifact repositories, thereby reducing the likelihood of accidental credential exposure at its **earliest possible stage**.

Verification and Ongoing Validation:

To verify remediation, existing build artifacts, container images, and packaged binaries should be scanned to ensure that no embedded Git tokens or credentials are present. Credential rotation should be validated by confirming that previously exposed tokens have been revoked and no longer grant access to source code repositories or CI/CD platforms. Ongoing validation should include continuous secret scanning of source code, build artifacts, and container images as part of CI/CD workflows. Access logs from version control systems and CI/CD platforms should be monitored to detect anomalous usage patterns associated with developer or automation credentials. Secret management policies, artifact scanning mechanisms, and preventive controls should be consistently enforced to ensure that embedded credentials cannot be reintroduced into the development or build lifecycle.